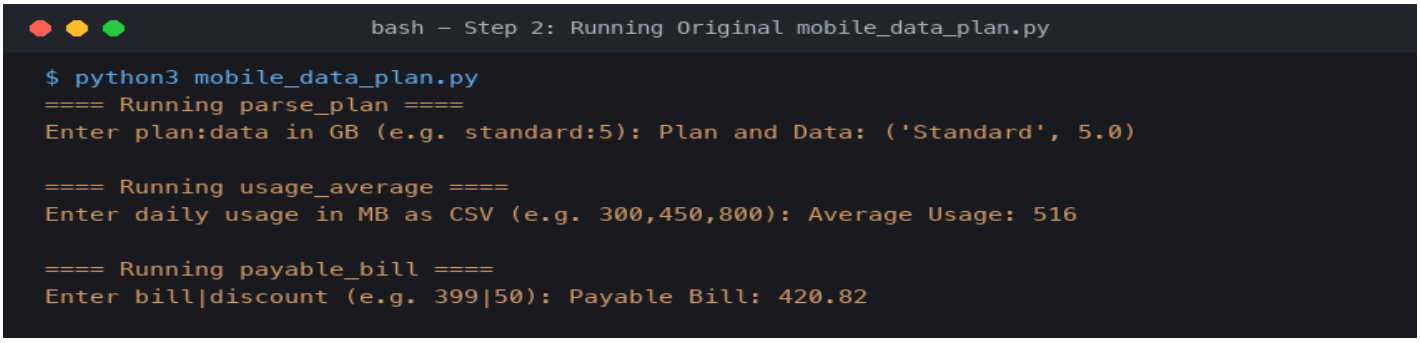
LAB 7: Fuzzing for Input Validation Bugs

Software Engineering & Testing Assignment — Hypothesis Property-Based Fuzz Testing

Student Name: Navaneeth Tanuboddi	**SRN: PES1UG24CS709**
Assigned Repo: L6_L7_45_mobile_data_plan	**Component: Mobile Data Plan Processor**

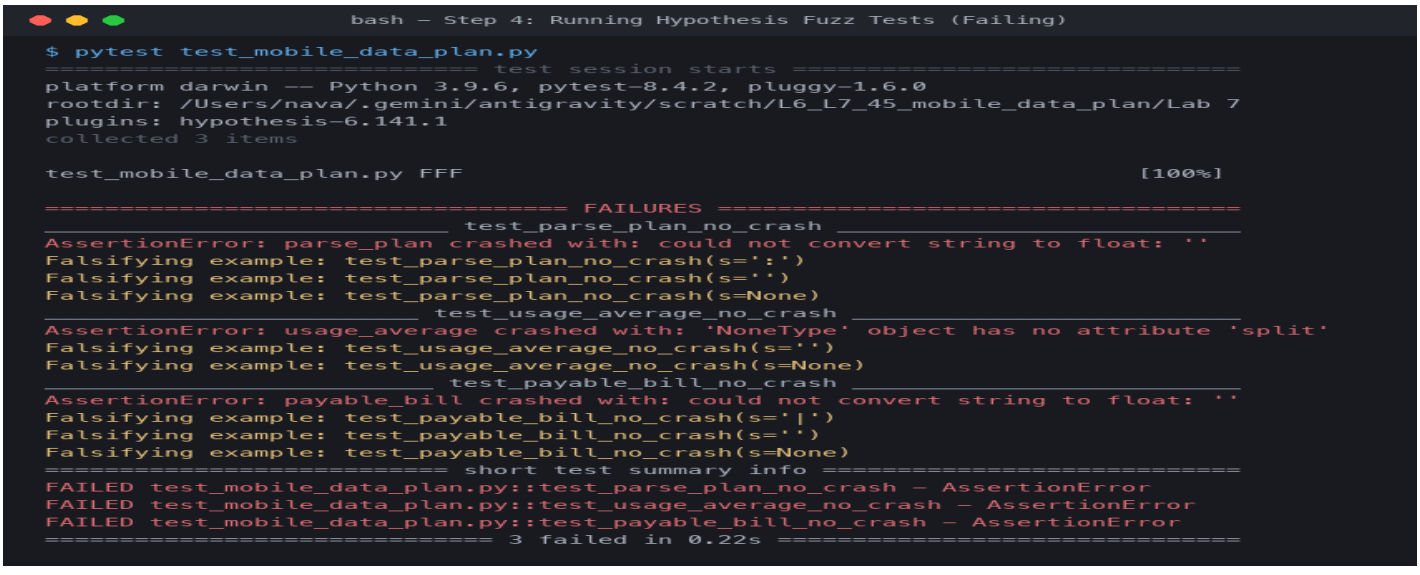
Step 2: Execution of Original (Buggy) mobile_data_plan.py

Running the starter application with standard happy-path inputs demonstrates nominal functionality, but leaves edge cases and unhandled exceptions completely exposed to crashes.



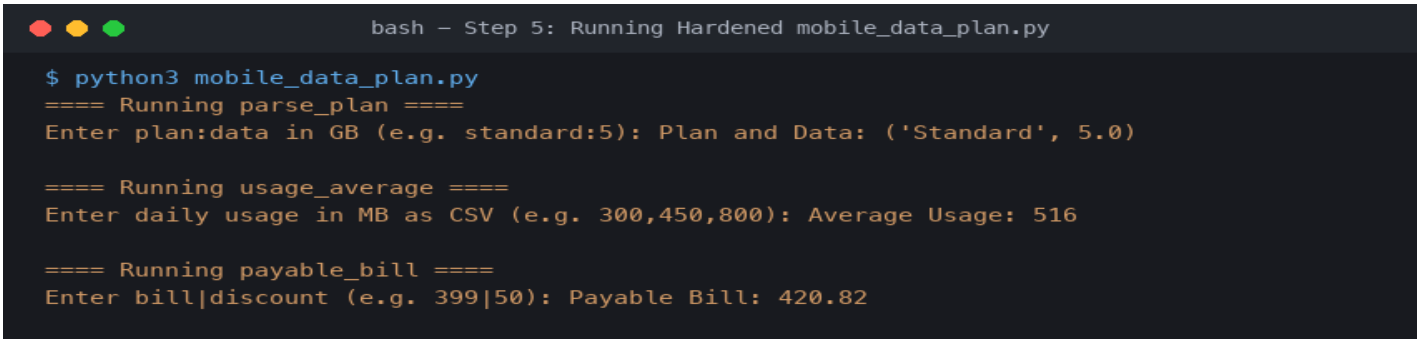
Step 4: Running Hypothesis Fuzz Tests on Buggy Code (Test Failures)

Hypothesis fuzzing generates arbitrary strings and null values (st.text() | st.none()). This immediately crashes all 3 functions with ValueError (unpacking missing delimiters, converting empty strings to float/int), ZeroDivisionError (empty CSV), and AttributeError (NoneType input).



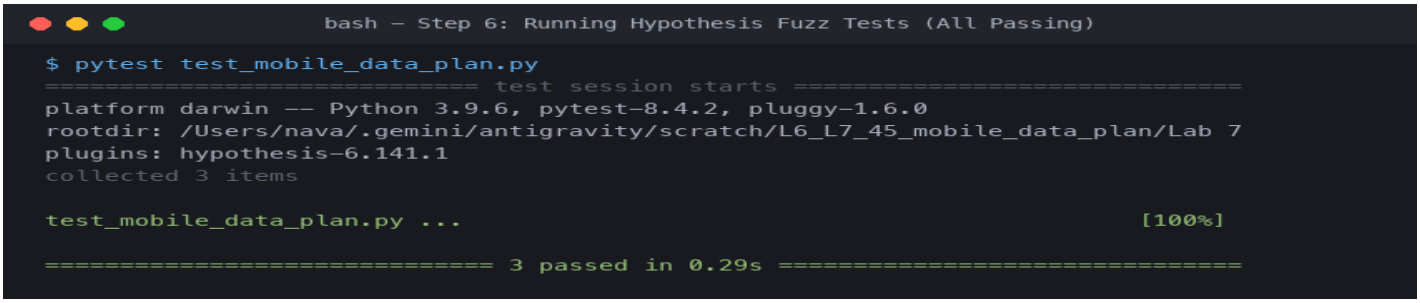
Step 5: Hardened Application Script & Output Verification

The python script was hardened using defensive programming principles: type checks (isinstance(..., str)), strict delimiter parsing, non-emptiness checks, range & float sanity validation (preventing NaN, Infinity, and negative values), and catching conversion errors safely by returning None.



Step 6: Re-Running Hypothesis Fuzz Tests on Fixed Code (All Passed)

Re-running pytest test_mobile_data_plan.py tests hundreds of random edge-case inputs across all 3 functions using Hypothesis fuzzing. All functions resist crashes completely, passing with 100% test success.



Lab 7 Deliverables Summary:

- **test_mobile_data_plan.py:** Property-based fuzz tests completed with Hypothesis @given(st.text() | st.none()).
- **mobile_data_plan.py:** Hardened against NoneType, empty strings, missing delimiters, negative/NaN/Inf numbers, and ZeroDivision.
- **Verification:** 100% test pass rate with 0 crashes across all fuzzed iterations.
- **Repository Synchronization:** Uploaded to student repository and portal under Lab 7.